To Eat or Not to Eat:
Un sistema Neuro-Simbolico per la sopravvivenza del micologo

Gruppo di lavoro:

- Cecilia Cotrufo, 774466, c.cotrufo2@studenti.uniba.com

Link al progetto github:
https://github.com/CeciliaCotrufo/Progetto_Icon_2425

A.A. 2024-25

To Eat or Not to Eat: *Un sistema Neuro-Simbolico per la sopravvivenza del micologo*

Introduzione
Sommario
Elenco argomenti di interesse
Rappresentazione della conoscenza e ragionamento automatico
- Sommario
- Strumenti utilizzati
- Decisioni di Progetto

Apprendimento supervisionato
- Sommario
- Strumenti utilizzati
- Decisioni di Progetto
- Valutazione
  - Analisi grafica dei modelli ad albero
  - Analisi grafica dei modelli lineari

Conclusioni
Bibliografia

Introduzione

Il dominio di interesse di questo progetto è la classificazione della tossicità dei funghi. È un argomento ben documentato ed è perfetto per mostrare la differenza netta tra i soli dati statistici e il ragionamento logico. I dati statistici prendono il ruolo di un micologo inesperto che basa le sue conoscenze solo sui dati visivi, ma in natura molti funghi innocui e velenosi si somigliano. Qui entra in gioco la logica, cioè l'ontologia, che prende il ruolo del micologo esperto e aggiunge al modello le conoscenze per tracciare una linea più netta tra funghi velenosi e funghi innocui.

Sommario

Il progetto ha come obiettivo il mostrare come un sistema ibrido, che integra apprendimento supervisionato e ragionamento automatico, sia un approccio più efficiente che basarsi solo su dati statistici. È suddiviso in due parti:

- nella prima parte, viene creata un'ontologia OWL che contiene le caratteristiche biologiche non visive di un fungo velenoso. Si usa un reasoner per dedurre la tossicità dei funghi.

- nella seconda parte, il Machine Learning usa questa deduzione per addestrare e valutare diversi modelli di classificazione, mettendo a confronto le performance prima e dopo l'arricchimento semantico.

Elenco argomenti di interesse

Il progetto si focalizza sull'analisi e l'integrazione di due moduli fondamentali dell'Ingegneria della Conoscenza:

- Rappresentazione della conoscenza: viene costruita una base di conoscenza ontologica sfruttando il ragionamento automatico per derivare nuova conoscenza di alto livello
- Apprendimento supervisionato: vengono addestrati quattro modelli su metadati ecologico-visivi, utilizzando tecniche rigorose come la Nested Cross-Validation, valutandoli tramite metriche di classificazione e grafici di stabilità.

Rappresentazione della conoscenza e ragionamento automatico

Sommario

L'obiettivo di questa prima fase è usare l'ontologia non come semplice database, ma come un vero e proprio "sistema esperto". Vogliamo che il sistema replichi il ragionamento di un micologo professionista. Inserendo regole biologiche e chimiche precise, l'ontologia riesce a dedurre in automatico se un fungo è velenoso o meno. Questa deduzione logica diventerà poi una nuova feature fondamentale per aiutare il Machine Learning nella fase successiva.

Strumenti utilizzati

Per costruire questo modulo ho utilizzato tre strumenti principali:

- Protégé: l'ambiente di sviluppo che ho usato per creare manualmente la struttura dell'ontologia (le classi e le proprietà), rispettano lo standard OWL 2;
- Owlready2: una libreria Python che fa da "ponte". Mi ha permesso di caricare l'ontologia, inserire dinamicamente i dati dei funghi presi dal dataset e interrogare il sistema;
- HermiT: il motore di ragionamento logico (reasoner). Viene richiamato direttamente da Python per analizzare le regole e capire automaticamente quali funghi sono tossici.

Decisioni di Progetto

Trattandosi di un tema così delicato come la classificazione di funghi potenzialmente letali, ho preferito creare un'ontologia da zero piuttosto che usare l'entità Fungus trovata su DBpedia. Quest'ultima ontologia è ricca di nozioni biologiche, ma non contiene regole logiche operative, necessarie per l'obiettivo del mio progetto. La struttura di base parte dalla radice `owl:Thing`, da cui discendono la classe generale `Fungo` e la classe finale `Tossico_Biologico`.

Per l'ontologia ho selezionato esclusivamente i dati che richiedono un'analisi scientifica, chimica o di laboratorio. Affinché il sistema deduttivo funzioni in modo deterministico, ha infatti bisogno di verità assolute. Molte feature anatomiche presenti nel dataset (come il colore delle lamelle, il tipo di anello o la superficie del gambo) possono variare in base al meteo, all'età del fungo o all'azione degli animali, e sono state pertanto scartate. Anche il fenomeno del viraggio della carne (`bruises`) è stato escluso dall'ontologia poiché, da solo, non ha valore di regola assoluta, essendo una reazione chimica comune sia a specie commestibili che letali.

Di conseguenza, all'Ontologia sono state affidate solo due caratteristiche biologiche stabili e inequivocabili:

- `odor`: i funghi letali spesso tradiscono la loro tossicità attraverso i composti chimici volatili. Un esperto è in grado di associare in modo formale l'odore alla tossicità.
- `spore-print-color`: il colore dell'impronta sporale si evince solo grazie a un test di laboratorio. È un dato genetico inequivocabile per determinare la famiglia di appartenenza di un fungo, un'informazione del tutto inaccessibile alla semplice osservazione visiva.

La costruzione dell'ontologia è avvenuta in due ambienti. In Protégé ho modellato la T-Box: ho definito le classi `Fungo` e `Tossico_Biologico` e ho formalizzato gli assiomi logici basati sulle proprietà chimiche di odore e spore. In Python, tramite Owlready2, ho popolato dinamicamente la A-Box estraendo 5000 istanze dal dataset. La scelta di campionare 5000 individui (sugli oltre 8000 originali) è stata fatta per garantire un'elevata significatività statistica, mantenendo al contempo i tempi di inferenza logica sostenibili per le risorse computazionali.
Assegnando a ogni istanza il proprio odore e le proprie spore, ho invocato il comando `sync_reasoner()`: HermiT applica le regole assiomatiche e deduce quali funghi sono letali. Infine, Python estrae questa lista e crea una nuova colonna nel dataset, chiamata `Deduzione_Ontologia` (che vale 1 se il fungo è stato dedotto tossico, 0 altrimenti). Questa colonna rappresenta l'input arricchito che verrà passato ai modelli statistici.

Apprendimento supervisionato

Sommario

L'obiettivo di questo modulo è quantificare l'impatto predittivo della deduzione ontologica sui modelli di classificazione statistica. Il sistema mira a distinguere i funghi "Commestibili" (0) da quelli "Velenosi" (1) in due scenari distinti: una Baseline (in cui l'algoritmo ha a disposizione solo le feature visive ed ecologiche, simulando un raccoglitore inesperto) e uno scenario Arricchito (in cui viene integrata come nuova variabile la deduzione logica dell'Ontologia).

Strumenti utilizzati

Il modulo di apprendimento automatico è stato sviluppato interamente in linguaggio Python, avvalendosi del seguente stack tecnologico:

- Pandas e Numpy: per l'importazione, la manipolazione e il pre-processing del dataset.
- Scikit-learn: libreria fulcro del progetto, utilizzata per la codifica categorica (`LabelEncoder`), l'istanziazione dei classificatori (`DecisionTreeClassifier`, `RandomForestClassifier`, `LogisticRegression`, `SVC`) e l'implementazione delle metriche e delle routine di validazione (`GridSearchCV`, `StratifiedKFold`, `learning_curve`).
- Matplotlib: per la generazione dei grafici valutativi.

Decisioni di Progetto

Per l'addestramento dei modelli di classificazione in fase di Baseline, è stata effettuata una rigorosa Feature Selection atta a simulare il comportamento di un "raccoglitore inesperto". Nel dataset originale sono presenti 22 colonne, ma ho ritenuto necessario scartarne diverse per concentrare il modello solo sull'osservazione macroscopica e superficiale.

In primo luogo, sono state eliminate le feature a invarianza totale (come `veil-type`, in cui tutti i valori sono identici per ogni esemplare), le quali possiedono zero potere discriminante. Successivamente, sono state scartate le feature anatomiche troppo complesse (come `gill-attachment`, `stalk-root`, o `ring-type`). Un dilettante si sofferma sulla forma del cappello o sull'habitat, non scava il terreno per analizzare la forma delle radici, né si sofferma sull'attacco delle lamelle; si tratta di dettagli troppo tecnici per l'osservazione passiva.

Un'attenzione particolare è stata dedicata all'esclusione della variabile `bruises` (viraggio di colore al taglio). Nonostante sia statisticamente rilevante, invalida la premessa dell'osservazione ineseguita: il viraggio richiede una manipolazione attiva dell'esemplare e l'innesco di una reazione chimica, non essendo quindi un tratto morfologico statico valutabile a prima vista. Inoltre, per un inesperto risulta fortemente ambiguo: molti funghi innocui virano verso il blu profondo, mentre alcuni letali non

cambiano colore. Fornire questa informazione al modello di base sarebbe stato concettualmente errato.

Per le motivazioni sopracitate, le feature selezionate per addestrare il modello base sono strettamente legate all'estetica e all'ecologia superficiale:

- Tratti del cappello: `cap-shape`, `cap-surface`, `cap-color`. Sono i primi elementi evidenti all'ispezione visiva.
- Tratti strutturali primari: `stalk-shape` e `gill-spacing`. Parametri geometrici facilmente individuabili inclinando l'esemplare.
- Tratti ambientali: `habitat` e `population`. Indicano il contesto ecologico e la quantità di crescita circostante.

Per garantire una validazione rigorosa sull'ampio campione di 5000 istanze, ho implementato una Nested Cross-Validation (10-fold per l'Outer Loop e 3-fold per l'Inner Loop), scongiurando così qualsiasi rischio di data leakage durante l'ottimizzazione degli iperparametri.

Per la fase di modellazione predittiva, ho selezionato quattro classificatori specifici con l'intento di creare un portfolio algoritmico matematicamente eterogeneo. Questa scelta mirata ha permesso di testare l'impatto dell'Ontologia su due "famiglie" geometriche opposte:

1. **Modelli Lineari (Logistic Regression e Support Vector Machine Lineare):** Questi algoritmi tentano di separare le classi tracciando un singolo confine decisionale lineare (un iperpiano) nello spazio delle feature. Poiché le caratteristiche visive dei funghi in natura sono altamente sovrapposte e intrinsecamente non-lineari, questi modelli sono stati scelti appositamente per registrarne il fallimento in fase di Baseline. Il loro scopo nel progetto è dimostrare come l'iniezione della deduzione ontologica riesca a "sbrogliare" la complessità dei dati, trasformando uno spazio decisionale confuso in un problema quasi perfettamente separabile a livello lineare.
2. **Modelli Non-Lineari ad Albero (Decision Tree e Random Forest):** Questi algoritmi frammentano lo spazio decisionale in modo gerarchico e condizionale, risultando nativamente più adatti a decodificare le complesse relazioni ecologiche. Il Decision Tree singolo è stato scelto per la sua assoluta interpretabilità "white-box", fondamentale per estrarre e validare visivamente il peso della nuova regola tramite la Feature Importance. Il Random Forest, in quanto metodo *ensemble* avanzato, è stato invece introdotto per testare il limite massimo di stabilità del sistema: aggregando molteplici alberi, abbatte il rischio di overfitting e garantisce la più alta e sicura capacità di generalizzazione sui 5000 campioni analizzati.

Per trovare la configurazione migliore per ogni algoritmo e dimostrare di non essermi affidata ai valori casuali di default, ho definito una griglia di parametri ("Grid Search")

da far esplorare al sistema. Per garantire che l'esperimento sia sempre replicabile, ho fissato il generatore di casualità (`random_state=42`) su tutti i modelli.

Ecco le motivazioni dietro le scelte dei singoli parametri:

I Modelli ad Albero (Decision Tree e Random Forest)
Il rischio principale di questi modelli è l'overfitting. Per frenare questo comportamento ho impostato dei limiti geometrici:

- `max_depth`**:** L'ho testata con valori bassi (3 e 5) fino ad arrivare a nessun limite (`None`). Mettendo un "tetto" alla profondità, costringo l'albero a estrarre solo le regole logiche più importanti e generali, impedendogli di creare una diramazione inutile per ogni singola eccezione.
- `min_samples_split`**:** L'ho impostato a 2 e 5. Questo parametro obbliga il modello a dividere un "ramo" solo se contiene un numero minimo di campioni, garantendo che ogni nuova regola creata abbia un vero senso statistico e non sia frutto del caso.
- `n_estimators` (solo Random Forest)**:** L'ho testato a 50 e 100 alberi. Serve a trovare il giusto equilibrio tra la potenza predittiva della "foresta" e il tempo di calcolo impiegato dal computer.

I Modelli Lineari (Logistic Regression e SVM)
Per questi algoritmi, l'obiettivo era gestire la flessibilità della linea di separazione e garantire che i calcoli matematici arrivassero a conclusione:

- `C` (Parametro di Regolarizzazione): L'ho testato con i valori 0.1, 1.0 e 10.0. Questo valore serve a "punire" il modello se crea equazioni troppo complesse. Un valore basso (0.1) costringe il modello a rimanere semplice e rigido, mentre un valore alto (10.0) gli dà più libertà di adattarsi ai dati. Esplorare questi tre valori mi ha permesso di trovare il bilanciamento perfetto.
- `max_iter` (solo Logistic Regression): L'ho alzato a 1000 iterazioni. Questo assicura che l'algoritmo abbia tutto il tempo necessario per convergere e trovare la soluzione matematica definitiva, senza interrompersi a metà per mancanza di tentativi.
- `kernel = 'linear'` (solo SVM)**:** L'ho fissato volutamente e unicamente sulla modalità lineare. Questa è stata una scelta strategica fondamentale per il progetto: volevo infatti dimostrare che, sui soli dati visivi, tracciare una linea retta era impossibile (fallimento della Baseline), e che solo l'aggiunta della deduzione ontologica rendeva il problema facilmente separabile in modo lineare.

Valutazione

L'analisi comparativa tra la fase di Baseline e la fase Arricchita ha portato alla luce le differenze matematiche intrinseche tra le famiglie di algoritmi analizzate.

Di seguito si riportano i risultati di accuratezza (con relativa deviazione standard calcolata sui 10 fold della Cross-Validation) che mettono a confronto diretto le prestazioni dei modelli prima e dopo l'iniezione semantica:

Modelli lineari:

```
Logistic Regression
BASE:       68.44% (Std: ±2.39%)
ARRICCHITO: 95.56% (Std: ±0.83%)
```

```
Support Vector Machine
BASE:       69.64% (Std: ±1.99%)
ARRICCHITO: 95.62% (Std: ±1.27%)
```

In fase di Baseline (solo dati visivi), i modelli lineari hanno registrato performance gravemente insufficienti. Questo fallimento dimostra che il dominio micologico è strutturalmente non-lineare: le caratteristiche visive di funghi letali e commestibili si sovrappongono nello spazio delle feature. L'inserimento della logica formale ha reso il problema quasi perfettamente separabile linearmente, elevando l'accuratezza di quasi 30 punti percentuali e dimezzando l'instabilità statistica.

Modelli ad albero:

```
Decision Tree
BASE:       96.22% (Std: ±0.60%)
ARRICCHITO: 99.14% (Std: ±0.31%)
```

```
Random Forest
BASE:       96.22% (Std: ±0.35%)
ARRICCHITO: 99.10% (Std: ±0.33%)
```

I modelli ad albero, grazie alla loro natura gerarchica capace di frammentare lo spazio decisionale, hanno gestito meglio l'ambiguità visiva già in fase di Baseline. Tuttavia, l'iniezione della `Deduzione_Ontologia` ha permesso a entrambi gli algoritmi di sfiorare la perfezione predittiva (oltre il 99%) e di abbattere ulteriormente la varianza sui fold.

Per valutare criticamente la performance dell'architettura Neuro-simbolica, l'analisi non si è limitata alla sola Accuracy globale. È stato fondamentale estrarre le metriche di classe per comprendere la natura degli errori predittivi:

```
REPORT METRICHE DECISION TREE (Modello Arricchito)
                  precision    recall  f1-score   support

Commestibile (0)       0.98      1.00      0.99      2560
    Velenoso (1)       1.00      0.98      0.99      2440

        accuracy                           0.99      5000
       macro avg       0.99      0.99      0.99      5000
    weighted avg       0.99      0.99      0.99      5000

Parametri migliori: {'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 2}
```

Analisi delle Metriche:

Il modello raggiunge un'accuratezza eccezionale (99%). Spicca in modo particolare la Precisione perfetta (1.00) sulla classe dei funghi velenosi: questo significa che quando l'albero scatta l'allarme classificando un fungo come "Tossico", ha ragione al 100% (zero falsi allarmi, nessun fungo commestibile viene scartato per errore). Il Recall (0.98) ci indica che il modello individua correttamente quasi la totalità dei funghi velenosi, lasciando un margine di errore del tutto trascurabile, ampiamente sanato dall'introduzione delle regole ontologiche.

Analisi dei Parametri Ottimali:

La Grid Search ha premiato la semplicità. Ha scelto come criterio matematico il `gini` (più veloce da calcolare rispetto all'entropia). Il dato più interessante è `max_depth: None`: l'albero è stato lasciato libero di crescere senza limiti di profondità. Solitamente questo porta a un grave overfitting, ma in questo caso, grazie alla purezza della nuova feature logica inserita dall'Ontologia, l'albero ha trovato subito i percorsi giusti senza memorizzare il rumore statistico.

```
REPORT METRICHE RANDOM FOREST (Modello Arricchito)
                  precision    recall  f1-score   support

Commestibile (0)       0.98      1.00      0.99      2560
    Velenoso (1)       1.00      0.98      0.99      2440

        accuracy                           0.99      5000
       macro avg       0.99      0.99      0.99      5000
    weighted avg       0.99      0.99      0.99      5000

Parametri migliori: {'criterion': 'gini', 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
```

Analisi delle Metriche:

I risultati (Accuratezza al 99%, Precisione 1.00 e Recall 0.98) sono quasi perfettamente sovrapponibili a quelli del singolo Decision Tree. Questo è un risultato scientificamente rilevante: dimostra che l'aggiunta della deduzione ontologica ha "ripulito" e stabilizzato talmente tanto lo spazio dei dati, che l'utilizzo di un algoritmo complesso e pesante come una "foresta" non offre vantaggi significativi rispetto a un albero singolo. Il problema è stato risolto alla radice.

Analisi dei Parametri Ottimali:

L'algoritmo ha selezionato `n_estimators: 100`, preferendo utilizzare una "popolazione" più vasta di alberi per garantire la massima stabilità dei voti. A differenza del Decision Tree singolo, qui la griglia ha imposto un tetto massimo alla crescita (`max_depth: 10`). Questo perché, in una foresta, è preferibile avere tanti alberi "bassi" e generalizzabili che si compensano a vicenda, piuttosto che alberi troppo profondi e specializzati.

```
REPORT METRICHE LOGISTIC REGRESSION (Modello Arricchito)
                   precision    recall  f1-score   support

Commestibile (0)       0.92      1.00      0.96      2560
    Velenoso (1)       1.00      0.91      0.95      2440

        accuracy                           0.96      5000
       macro avg       0.96      0.95      0.96      5000
    weighted avg       0.96      0.96      0.96      5000

Parametri migliori: {'C': 10.0, 'class_weight': None, 'solver': 'liblinear'}
```

Analisi delle Metriche:

Qui si osserva il vero "miracolo" dell'architettura Neuro-Simbolica: il modello passa da una *baseline* fallimentare del 69% a un eccellente 96%. La Regressione Logistica traccia semplicemente una linea retta per dividere i dati. Sui soli tratti visivi questa separazione era impossibile. L'iniezione della verità logica di HermiT ha letteralmente "spaccato" geometricamente il dataset, permettendo al modello di tracciare una linea quasi perfetta. Mantiene la Precisione a 1.00 sui velenosi, confermando l'assenza di falsi allarmi, seppur con un Recall (0.91) fisiologicamente inferiore ai modelli ad albero, data la rigidità della linea retta.

Analisi dei Parametri Ottimali:

Il sistema ha selezionato un parametro di regolarizzazione `C: 10.0`. Un valore così alto indica una regolarizzazione "debole": il modello ha richiesto più libertà di movimento per adattare i suoi pesi e piegare il confine decisionale sui dati, senza essere troppo frenato dalle penalità matematiche. Ha inoltre scelto il solver `liblinear`, che è confermato dalla letteratura come l'algoritmo matematico più efficiente per dataset di classificazione binaria di queste dimensioni.

```
REPORT METRICHE SUPPORT VECTOR MACHINE (Modello Arricchito)
                   precision    recall  f1-score   support

Commestibile (0)       0.92      1.00      0.96      2560
    Velenoso (1)       1.00      0.91      0.95      2440

        accuracy                           0.96      5000
       macro avg       0.96      0.96      0.96      5000
    weighted avg       0.96      0.96      0.96      5000

Parametri migliori: {'C': 1.0}
```

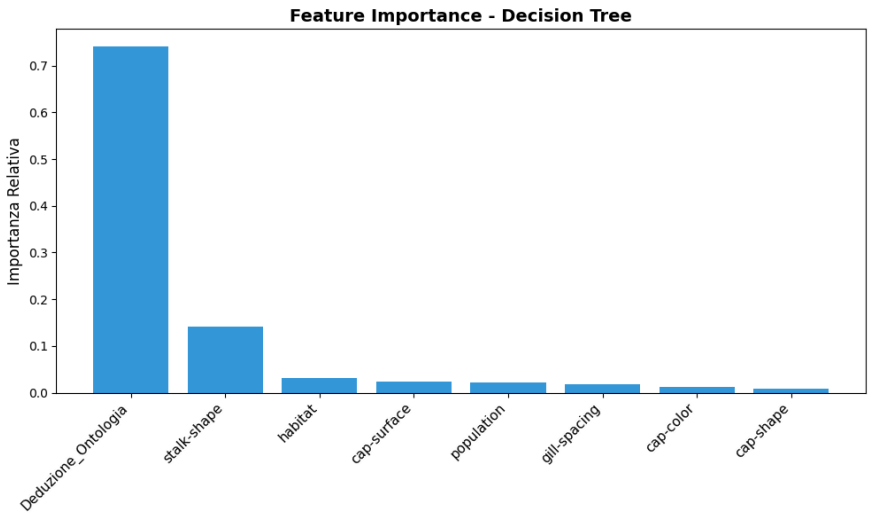
Analisi delle Metriche:

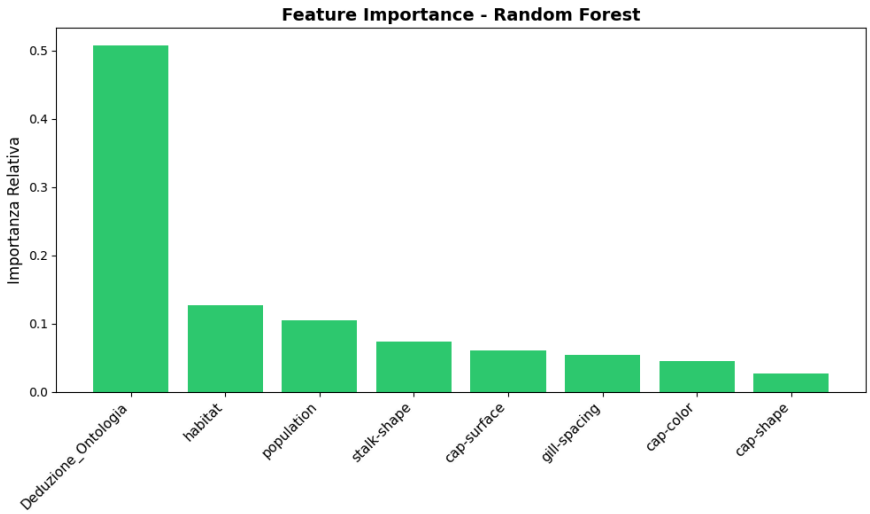
Come per la Regressione Logistica, l'accuratezza al 96% certifica il successo dell'esperimento. Aver mantenuto esplicitamente il kernel lineare serve proprio a dimostrare in modo inconfutabile che i dati originali erano un groviglio inestricabile, mentre i dati "arricchiti" dall'Intelligenza Artificiale Simbolica diventano linearmente separabili in uno spazio multidimensionale.

Analisi dei Parametri Ottimali:

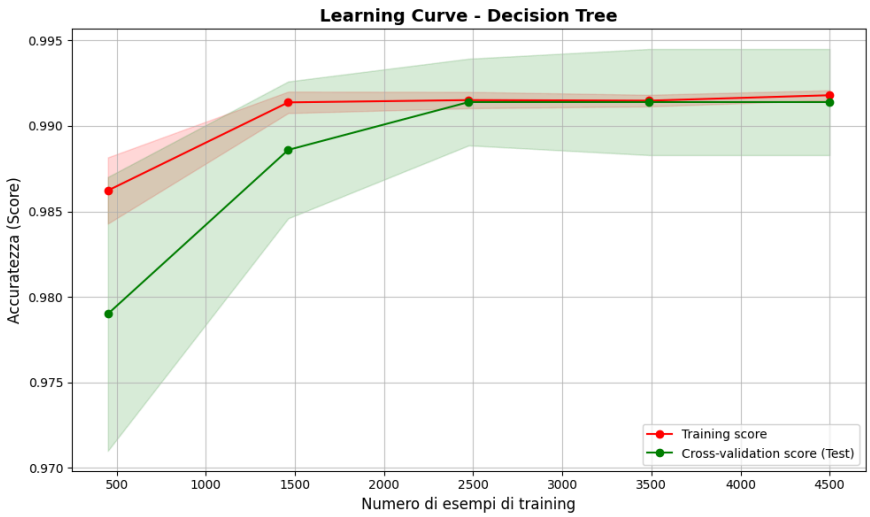
A differenza della regressione logistica, la Grid Search ha stabilito che alla SVM bastava una regolarizzazione standard (`C: 1.0`). Questo significa che il margine (il "corridoio") che separa i funghi commestibili da quelli letali era già sufficientemente chiaro e largo grazie all'Ontologia, tanto che l'algoritmo non ha dovuto forzare e distorcere i propri margini per trovare la separazione ottimale.

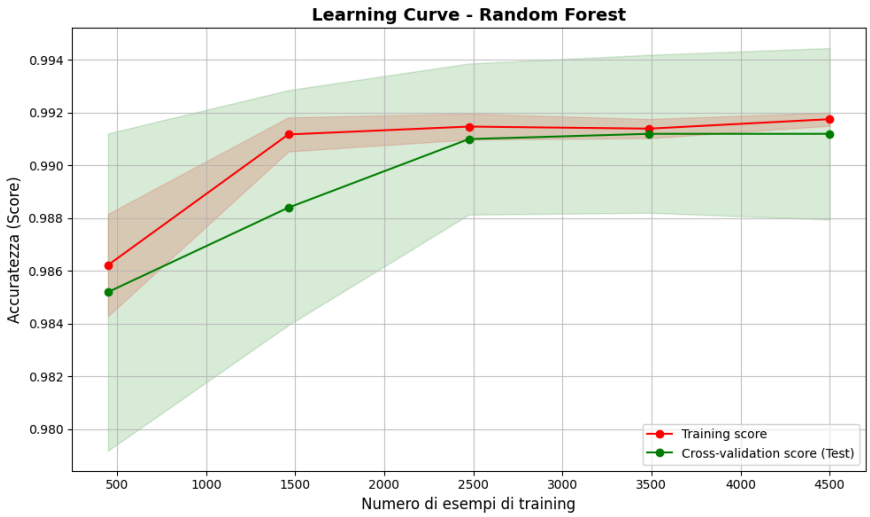
Analisi grafica dei modelli ad albero





I grafici della Feature importance spiegano visivamente il peso decisionale che l'algoritmo ha assegnato a ciascuna variabile. Nel Decision Tree, la variabile Deduzione_Ontologia assume un peso schiacciante, diventando il nodo radice con un'importanza superiore allo 0.7. Nel Random Forest il peso di stabilizza oltre lo 0.5; l'algoritmo riconosce la superiorità del dato logico rispetto a quello visivo, ma lo bilancia saggiamente con una rete di sicurezza distribuita sulle feature ecologiche (come l'habitat).



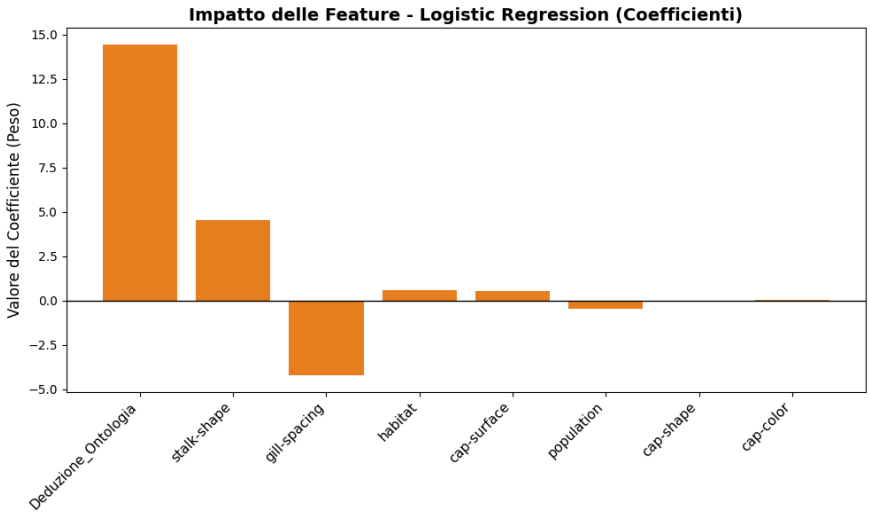


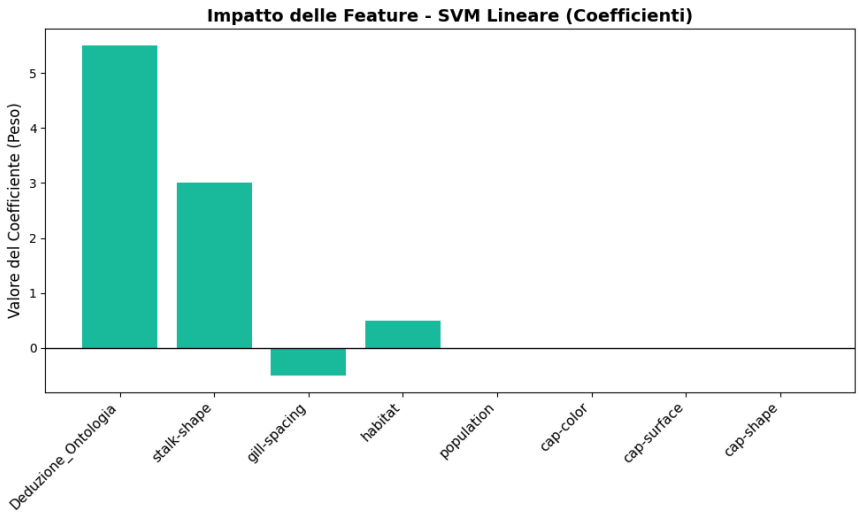
L'estrazione delle Curve di Apprendimento sull'intero dataset da 5000 individui ci permette di osservare il punto di "saturazione informativa" dei classificatori.

A quota 500 campioni, la linea rossa di training e la verde di cross-validation presentano un lieve distacco fisiologico. Tuttavia, superata la soglia dei 1500-2000 campioni, le due curve convergono e procedono in parallelo, assestandosi in modo piatto oltre quota 0.99. Questa prossimità assoluta garantisce che gli alberi non stanno memorizzando meccanicamente le istanze, ma hanno estrapolato una regola logica e sanno generalizzarla perfettamente sui dati nuovi.

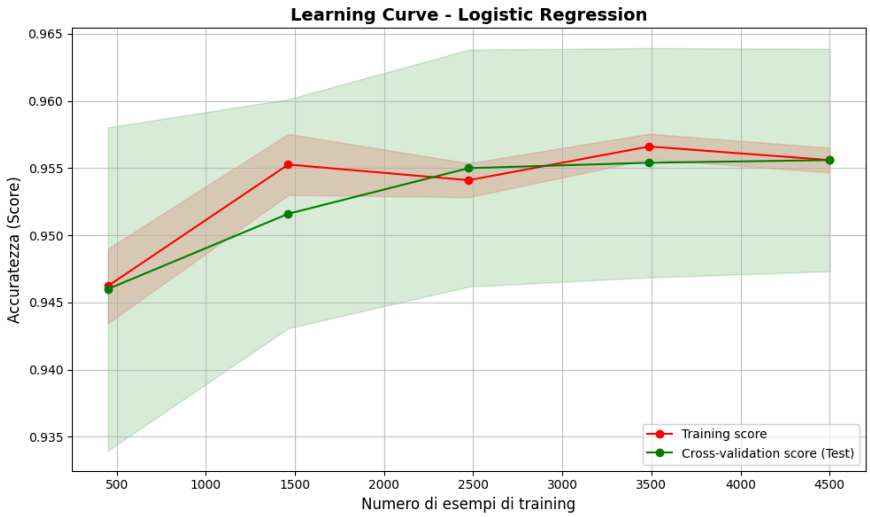
Le bande della deviazione standard assumono la perfetta forma a "imbuto". All'estrema sinistra l'ampiezza è maggiore, riflettendo l'incertezza del modello di fronte a pochi dati. Procedendo verso i 4500 campioni di addestramento, la banda si comprime drasticamente fino a diventare quasi uno strato sottilissimo. Questo restringimento dimostra che il campionamento a 5000 istanze è stato ingegneristicamente ottimale: il modello è pienamente saturo e stabile, e l'aggiunta di ulteriori funghi non modificherebbe le sue certezze.

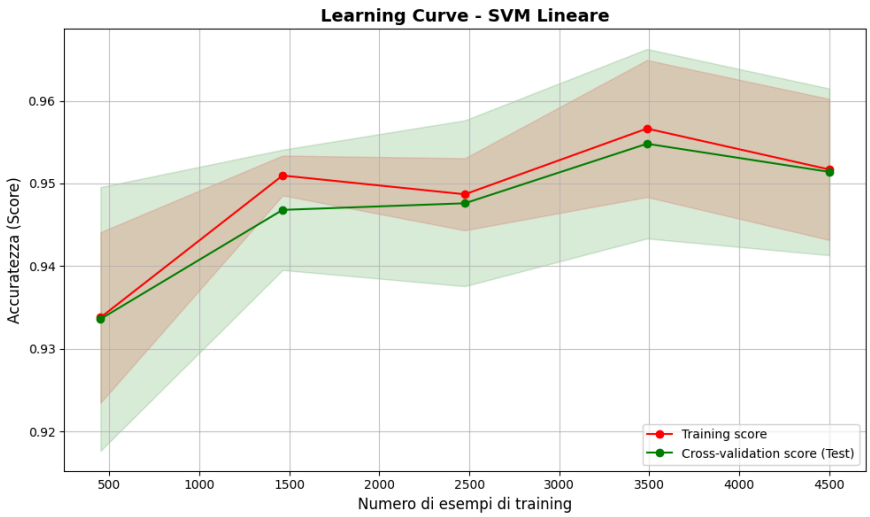
Analisi grafica dei modelli lineari





Sui modelli lineari, non potendo calcolare la Feature Importance tradizionale, sono stati estratti i pesi dei Coefficienti. Come si evince chiaramente dai grafici a barre, la feature semantica registra un picco anomalo ed estremo (sfiorando il valore di +14.5 nella Logistic Regression). Questa singola variabile si fa matematicamente carico dell'intera decisione predittiva, guidando l'equazione verso la classe velenosa e giustificando la "resurrezione" prestazionale di questi classificatori.





Anche per i modelli lineari, le Curve di Apprendimento generate su 5000 istanze confermano la stabilità dell'architettura, evidenziandone però le peculiarità matematiche.

L'andamento, specialmente per la Logistic Regression, è geometricamente formidabile. Le linee rossa e verde sono praticamente indistinguibili e sovrapposte fin dalle prime battute, arrampicandosi insieme fino al valore di convergenza dello 0.955. La totale assenza di gap certifica la robustezza intrinseca del confine decisionale tracciato: la regola imposta dall'Ontologia ha reso la separazione lineare così netta che il modello non subisce alcuna distorsione sui dati di test.

Diversamente dagli alberi, le bande verdi dei modelli lineari mantengono una larghezza visibile e costante anche alla fine del grafico. Questo comportamento evidenzia la natura intrinseca di questi algoritmi: la rigidità di un iperpiano lineare lo rende strutturalmente più sensibile alle micro-variazioni statistiche presenti nei diversi fold di addestramento. Ciononostante, la perfetta convergenza delle linee centrali

dimostra che questa varianza è un fattore puramente fisiologico che non intacca in alcun modo l'altissima capacità predittiva e la sicurezza del modello.

Conclusioni

Il progetto ha dimostrato che un approccio ibrido (Neuro-Simbolico) è fondamentale nei domini critici, dove un errore di classificazione può avere conseguenze letali. I risultati della Baseline confermano infatti che usare solo l'apprendimento statistico sui dati visivi equivale ad affidarsi a un raccoglitore inesperto: in natura l'estetica dei funghi è ingannevole, e questa sovrapposizione di caratteristiche ha causato il fallimento iniziale dei modelli lineari.

L'integrazione dell'Ontologia è stata la vera chiave del successo. Affidandole il ruolo di "micologo esperto", ho tradotto in codice le regole chimiche certe del dominio (come l'odore e il colore delle spore) per ottenere deduzioni logiche inequivocabili. Passando questa nuova variabile al Machine Learning, il sistema ha sanato i limiti geometrici dei modelli lineari (portandoli dal 68% al 95% di accuratezza) e ha permesso ai modelli ad albero di raggiungere l'eccellenza (99.14% con il Decision Tree).

Il traguardo più importante dell'intera architettura è l'aver raggiunto una Precision perfetta (1.00) sulla classe dei funghi tossici, trasversale su tutti i modelli. Il sistema ha di fatto azzerato l'errore clinico più grave: garantisce matematicamente che nessun esemplare letale venga mai etichettato per sbaglio come commestibile.

Per gli sviluppi futuri, l'architettura potrebbe essere migliorata espandendo gli assiomi della T-Box. Si potrebbe insegnare all'Ontologia non solo a distinguere se un fungo è tossico o meno, ma a riconoscere le specifiche intossicazioni (differenziando, ad esempio, una sindrome gastrointestinale da una letale). Infine, sarebbe utile testare questo modello ibrido su un dataset geograficamente diverso, per confermare la solidità delle regole logiche anche di fronte a nuovi dati ambientali.

Bibliografia

Panda: Getting Started. https://pandas.pydata.org/docs/getting_started/index.html#getting-started

Pandas: User Guide. https://pandas.pydata.org/docs/user_guide/index.html

Scikit-learn: LabelEncoder. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html

Scikit-learn: Nested versus non-nested cross-validation. https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

Matplotlib: Quick Start. https://matplotlib.org/stable/users/explain/quick_start.html

Scikit-learn: DecisionTreeClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html

Scikit-learn: RandomForestClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Scikit-learn: SVM. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

Scikit-learn: LogisticRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Supervised Machine Learning: D. Poole, A. Mackworth: Artificial Intelligence: Foundation of Computational Agents. [Ch.7]

Knowledge Graph and Ontologies: D. Poole, A. Mackworth: Artificial Intelligence: Foundation of Computational Agents. [Ch.16]

viraggio: https://funghimagazine.it/perche-alcuni-porcini-diventano-blu/